AI & Blackjack

Sasha Korol Michelangelo Prisciano Olivia Terris

Younis Al-Riyami Oleksii Popov

December 2025

Contents

1 Introduction

Blackjack ([2]) is a popular card game where several players play against the dealer and at each round they have the option to request another card (referred to as "Hit") or keep the ones they have and pass their turn (referred to as "Stand"). This is a recurring example in the AI literature ([3]) as it involves designing an agent that learns optimal behavior in a stochastic environment rather than a fully deterministic one. Our project considers a simplified version of the game with the following rules:

- The AI Agent is the only player (besides the Dealer)
- The Dealer plays a fixed strategy i.e. they play "Hit" if their sum is 17 or less and they "Stand" otherwise
- There is no betting, splitting, etc. If the Agent's current sum is greater than the Dealer's, the Agent wins and collects a reward. Otherwise, the Agent loses or draws.

The Dealer's hidden card as well as the composition of the deck is unknown, therefore decisions must be made without full knowledge of the environment. Our problem statement in the context of AI then becomes the following: how can we build an autonomous agent that learns the best behavior in an uncertain and probabilistic environment and is able to find an optimal policy to follow? Our project aims to find a solution to this problem with two different approaches i.e. Search and Reinforcement Learning.

2 Modeling the Problem

2.1 State-Space Formulation

The components of our state-space model for this problem are the following:

- `agent_current_sum`: this is the sum of the card's values that the agent currently holds. Its value $\in \{12, 13, \ldots, 21\} \rightarrow$ which adds up to 10 possible values
- `dealer_visible_card`: this is the card that the Dealer holds face up. Its value $\in \{1, 2, \ldots, 10\} \rightarrow$ which adds up to 10 possible values
- `is_ace_usable`: this is a boolean that tracks whether the Ace that the agent holds (if they hold one) is "usable" i.e. if its value can be 11 without making the Agent go bust. Its value $\in \{\text{true}, \text{false}\} \rightarrow$ which adds up to 2 possible values

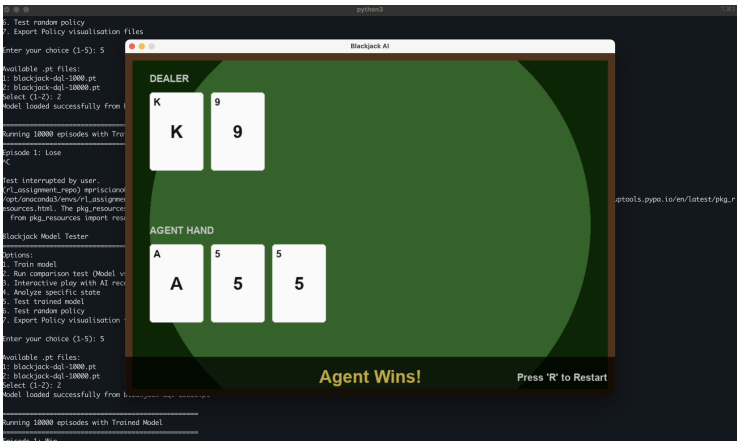
Therefore the size of the state space (excluding terminal states) is:

```
10 x 10 x 2 = 200
```

Rewards are assigned in the following way: +1 for wins, 0 for draws, and -1 for losses.

2.2 Gathering Data

With our model defined, the next step is choosing where to source data from to train our agents. One option would be to spend countless hours in a casino and manually recording every possible game state - but as diligent and resourceful students, we chose a more efficient approach. Instead, we built a custom environment with a dedicated UI capable of simulating games and keeping track of all states that occurred throughout each episode. The randomness in the environment is handled by Python's `random` library, which implements the Mersenne Twister algorithm (a pseudorandom number generator), well-suited for statistical modeling [6].



3 AI Strategies

3.1 Informed Search Method

This part aims to showcase how the algorithm worked, rather than playing blackjack effectively. Search algorithms can be limited when used in Blackjack because the game heavily relies on chance and the next move cannot be predicted deterministically. For this reason, we let the Search agent have some additional features: the agent can see the deck (therefore it knows what cards will come next) and the dealer's score before they reveal the second card.

The method tries to win while "hitting" (taking cards) as least as possible or just enough to draw with the dealer. It explores what the dealer will do if the agent "stands" (ends its turn) by creating copies of the agent's and dealer's hands and the deck for simulation. In our simplified version of Blackjack, after revealing their second card, the dealer must hit if their score is below 17 and stand if it is above that. Using this rule the method tries to make the simulation of the agent stand and see what the simulated dealer's score will be after both it and the dealer stand. The algorithm repeats this, each time having the simulated agent hit one time until it reaches the first point when the simulated dealer loses or when hitting again would result in a loss for the agent. If at any point the scores result in a draw, the algorithm saves that path in case it does not find a path that allows winning, in which case it gives the path to the agent to draw with the dealer. The path which the method returns is the name of the card the agent must get before standing.

3.2 Reinforcement Learning Method

Blackjack is a tricky game to model because it's a mixture of chance and skill. The approximations in the search method strategy made it possible to explore all states and come up with an effective strategy at each round. With the Reinforcement Learning approach we want to build an even more effective agent that can overcome those obstacles and play Blackjack while dealing directly with the chance-element present in the game. In the beginning there was a lot of research done on choosing which Reinforcement Learning strategy was the most suitable for an agent that learns how to play Blackjack. On the implementation side the main challenge was learning the Torch library in order to train the agent faster as well as experimenting with tweaking the available hyper-parameters (i.e. number of episodes during training, number of hidden layers, network sync rate, etc) to find the optimal configuration.

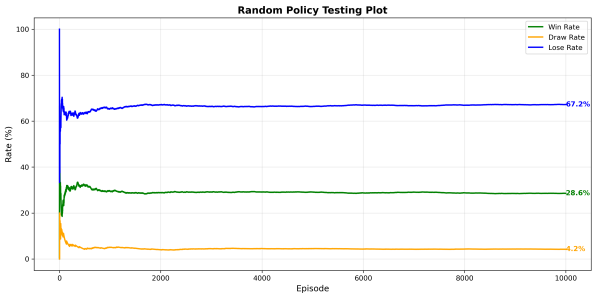
Within the Reinforcement Learning realm there are several approaches available, however the one that seemed to suit best the needs of this specific game scenario is Q-Learning, an approach first introduced by Christopher Watkins[1]. With this approach every pair `(state, action)` is associated to a **Q-value** which is as a measure of the expected future reward that the agent will collect if it takes that action. In our specific scenario the RL Agent will evaluate the two Q-values associated with the actions "Hit"/"Stand" and, once training is completed, will choose the action that has the higher expected reward, i.e. the higher Q-value.

With classic Q-Learning approach the policy is represented through a two dimensional array (also called "Q-Table") where rows represent available states, columns represent the available actions and each entry represents the Q-value of that action-state pair. This approach has some limitations due to the fact that the Q-Table size grows linearly with the number of possible states. Deep Q-Learning on the other hand approximates the optimal policy with a neural network trained over a number of episodes. In our case the input layer consists of the three nodes representing the features of the state-space model (`current_sum_of_agent_hand`, `dealer_card`, `is_ace_usable`) while the output layer consists of two nodes that represent the Q-value of the actions "Hit" and "Stand".

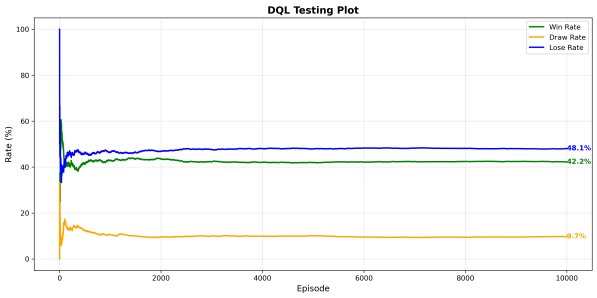
During training we used the Epsilon-Greedy exploration strategy to balance exploration and exploitation: in the beginning the agent predominantly chooses random exploration and, as epsilon decays, the agent increasingly relies on its learned policy to choose the best available action. **Experience Replay** [4] and **Double DQN** [5] were also used in an attempt to improve training stability.

We trained the agent in multiple scenarios, over a different number of episodes. Our analysis revealed that extended training periods typically led to better performance during testing. To evaluate the RL Agent's performance we benchmarked it against an agent that selected actions randomly and got back the

promising results shown below. In particular for the RL Agent Win Rate saw an increase of 19.1% and Lose Rate saw a decrease of 13.6%.



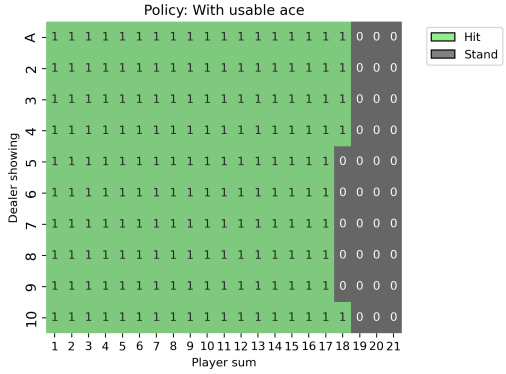
(a) Agent with a "random" policy



(b) Agent trained over 50000 episodes

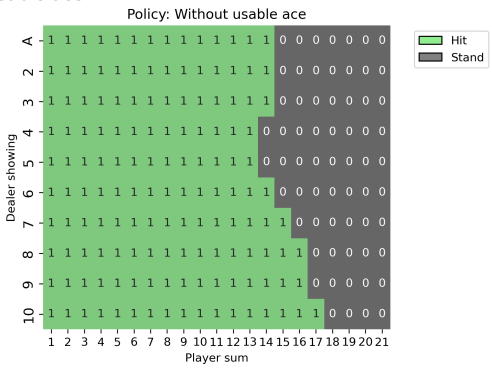
The following grids are the representations of the policy of an RL Agent trained over 50000 episodes.

With usable ace



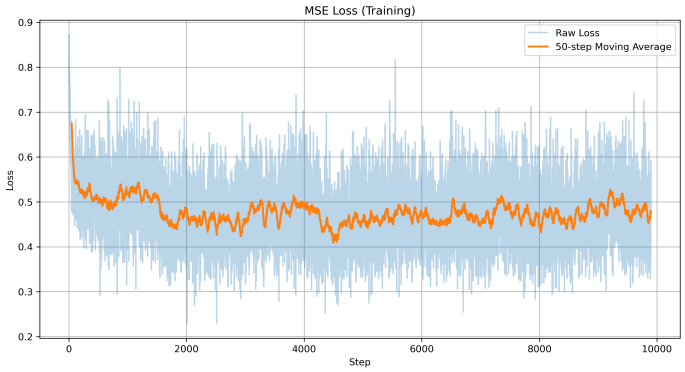
(a) "Usable Ace" Agent Policy

Without usable ace

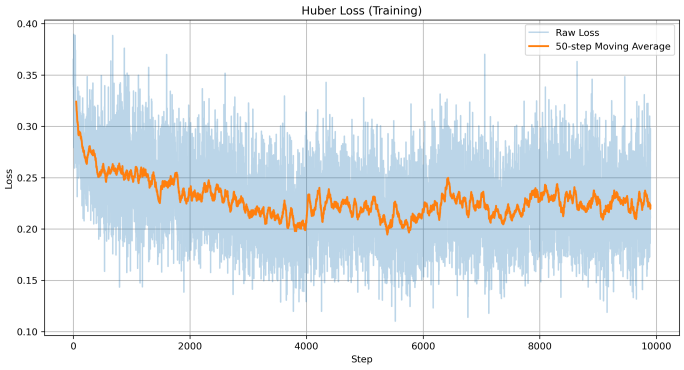


(b) "Without Usable Ace" Agent Policy

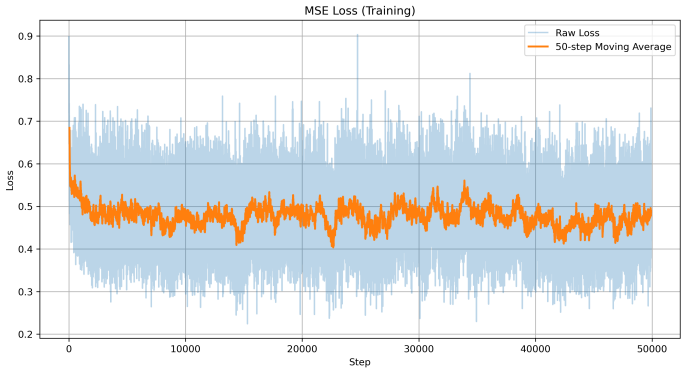
These results are very promising! The agent has learnt to recognise the high risk of going bust when the sum of its cards is close or equal to 21, appropriately choosing "Stand" in these cases. Interestingly, the agent plays more aggressively when holding a usable ace, selecting "Hit" in situations where it would otherwise "Stand". This demonstrates that the agent has discovered it can afford greater risk with a usable ace, since the ace value can be adjusted from 11 to 1 to avoid going bust. We can also see clearly from the "Without Usable Ace" grid that the RL Agent also learnt to adjust its strategy based on the dealer's showing card. When the dealer shows a high card the agent plays more aggressively and conversely when the dealer shows a low card the agent adopts a conservative approach, standing earlier to avoid going bust.



(a) Training using MSE Loss (over 10000 episodes)

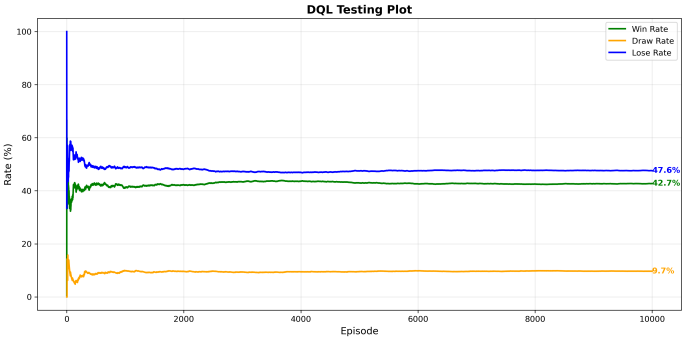


(b) Training using Huber Loss (over 10000 episodes)

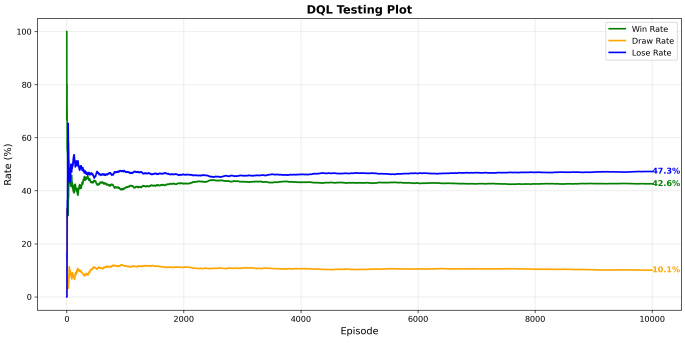


(c) Training using MSE Loss (over 50000 episodes)

(d) Training using Huber Loss (over 50000 episodes)



(e) Testing with model trained with MSE Loss



(f) Testing with model trained with Huber Loss

To evaluate whether the choice of loss function plays any role in the model performance, we compared MSE Loss and Huber loss function trend during training over 10000 and 50000 episodes. While both exhibited fluctuations, their values stayed within consistent bounds. Testing results in panels (e) and (f) confirm that loss function choice does not meaningfully affect model performance.

4 Results and Comparison

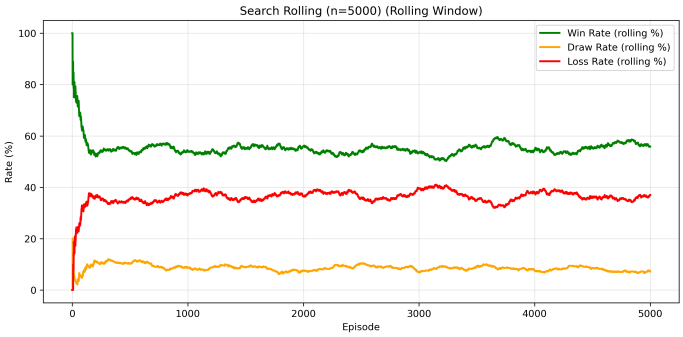
4.1 Theoretical Comparison

Reinforcement Learning agents learn directly from experience and naturally adapt to different rule variations or alternative reward structures. They don't always require explicit environment modeling, only reward feedback is needed for learning. On the other hand they require a substantial number of training episodes before stabilising, resulting in inconsistent early-stage performance. Convergence can be slow, particularly in complex or ambiguous environments.

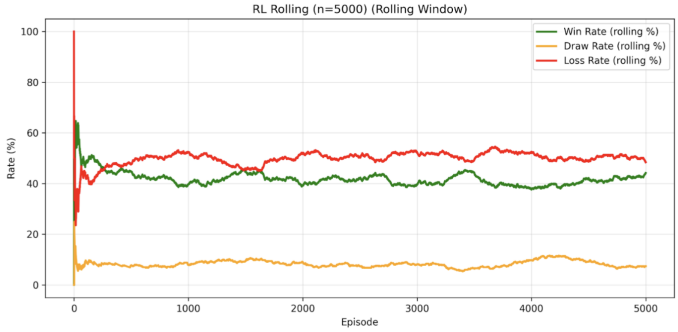
Search-based agents, on the other hand, often provide optimal solutions through a systematic exploration of the possible states/actions. Performance is usually consistent over time and there is no "training" period required, aside from the exploration part. On the other hand, they are less effective in more complex environments as the number of states can grow exponentially by adding more simple features into the state space.

4.2 Results Comparison

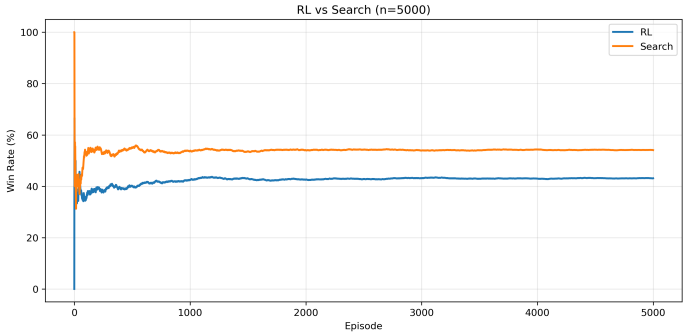
In our specific case the search-based agent has two major advantages over the RL agent. First, the environment has a relatively small state space of only 200 states. Second, and more significantly, the search agent knows the deck composition in advance and can plan deterministically, while the RL agent must learn through direct interaction with the stochastic environment. These factors suggest that theoretically the search agent should substantially outperform the RL agent: our experimental results confirmed this expectation.



(a) Search Method Testing Trend



(b) RL Method Testing Trend



(a) Search/RL Test Trends Comparison

4.3 Suggested Improvements

The current Reinforcement Learning configuration works well, but there are several factors that could be refined in the next iteration. The policy and target networks sync every 100 steps — a simple approach that's been effective so far but it might be worth exploring a smarter sync strategy. For example, we could define a custom metric that measures the divergence between the two networks by comparing the absolute gradient values, and trigger a sync between the networks only when that difference crosses a set threshold. There is also room to experiment with different loss functions or to run training over a much longer period to see how performance evolves over time.

With regards to the Search method, one possible improvement is to reduce the amount of data copied for simulation to enhance efficiency. This current version copies the entire state of the game into a simulation when in reality, the maximum amount of hits that can be done in a one-player vs dealer game is much less than a full deck of cards.

5 Conclusions

This project was a considerable team effort and a great way of putting into practice everything that we have learnt throughout the "Intro to AI" module. We learnt how to implement different AI methods to the same real-world problem and analyse weaknesses/strengths of each method, as well as how to analyse a model performance by plotting graphs and improve it where possible.

6 Above and Beyond

The following "above and beyond" elements have been already mentioned throughout the report. To reiterate, these are the ones we implemented:

- We selected a problem very different from navigation, with an inherent stochastic nature
- We used an informed search strategy
- We built a custom environment implementation with a dedicated UI
- We displayed graphs of results throughout the report
- All scripts have a terminal-based interface to interact with the user

References

[1] Christopher Watkins, *"Technical Note: Q-Learning"*, 1992.

[2] Blackjack Game Summary, `https://bicyclecards.com/how-to-play/blackjack`

[3] Sutton & Barto, 1998 *Reinforcement Learning: An Introduction*

[4] Mnih, Kavukcuoglu, Silver, Graves, Antonoglou, Wierstra, Riedmiller *Playing Atari with Deep Reinforcement Learning*

[5] Van Hasselt, Guez, Silver *Deep Reinforcement Learning with Double Q-learning*

[6] Python random() Documentation, `https://docs.python.org/3/library/random.html`